Blind Maze Search - BFS and DFS Comparison

This notebook demonstrates Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms for maze pathfinding. Answers Q: Blind Search to Play Maze

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt

class Node():
    """A search node class for Maze Pathfinding"""

    def __init__(self, parent=None, position=None):
        self.parent = parent
        self.position = position
        self.c = 0  # cost from source to current node

    def __eq__(self, other):
        return self.position == other.position

    def __hash__(self):
        return hash(self.position)


def search_path(maze, start, end, method='DFS'):
    """Returns a list of tuples as a path from the given start to the given

    # Create start and end node
    start_node = Node(None, start)
    start_node.c = 0
    end_node = Node(None, end)
    end_node.c = 0

    # Initialize both open and closed list
    open_list = []
    closed_list = set()

    # Add the start node
    open_list.append(start_node)

    # Loop until you find the end
    expanded_nodes=0
    queue_size=0
    while len(open_list) > 0:

        # Get the current node
        current_node = open_list[0]
        current_index = 0

        # Pop current off open list, add to closed list
        # depending on how the nodes are added to the queue, this will imple
```

```
        open_list.pop(current_index)
        closed_list.add(current_node)

        # Found the goal
        if current_node == end_node:
            path = []
            current = current_node
            while current is not None:
                path.append(current.position)
                current = current.parent
            return(expanded_nodes,queue_size,path[::-1])

        # Generate children
        expanded_nodes=expanded_nodes+1
        if(len(open_list)>queue_size):
            queue_size=len(open_list)
        children = []
        for new_position in [(0, -1), (0, 1), (-1, 0), (1, 0), (-1, -1), (-1

            # Get node position
            node_position = (current_node.position[0] + new_position[0], cur

            # Make sure within range
            if node_position[0] > (len(maze) - 1) or node_position[0] < 0 or
                continue

            # Make sure walkable terrain
            if maze[node_position[0]][node_position[1]] != 0:
                continue

            # Create new node
            new_node = Node(current_node, node_position)

            # Append
            children.append(new_node)

        # Loop through children
        for child in children:

            # Child is on the closed list
            if child in closed_list:
                continue

            # Create the updated cost values
            child.c = current_node.c + np.sqrt(np.square(child.position[0] -


            # Child is already in the open list
            childAlreadyExist=False
            for open_node in open_list:
                if child == open_node and child.c >= open_node.c:
                    childAlreadyExist=True
                    break

            # Add the child to the open list
            if(not childAlreadyExist):
```

```
                if method=='BFS':
                    open_list.append(child)
                if method=='DFS':
                    open_list.insert(0,child)

def pathLength(path):
    dis=0
    for i in range(len(path)-1):
        x1=path[i][0]
        y1=path[i][1]
        x2=path[i+1][0]
        y2=path[i+1][1]
        dis=dis+np.sqrt(np.square(x1-x2)+np.square(y1-y2))
    return(dis)

def run_maze_search():
    """Main function to run maze search with BFS and DFS"""

    maze = [[0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]

    print("Maze:")
    for row in maze:
        print(row)

    start = (1, 0)
    end = (0, 1)

    print("\n" + "="*50)
    print("SHOWING RESULTS FOR BFS:")
    print("="*50)
    expanded_nodes_bfs, queue_size_bfs, path_bfs = search_path(maze, start,
    print(f"Path length: {pathLength(path_bfs)}")
    print(f"Expanded nodes: {expanded_nodes_bfs}")
    print(f"Maximum queue size: {queue_size_bfs}")
    print(f"Path: {path_bfs}")

    print("\n" + "="*50)
    print("SHOWING RESULTS FOR DFS:")
    print("="*50)
    expanded_nodes_dfs, queue_size_dfs, path_dfs = search_path(maze, start,
    print(f"Path length: {pathLength(path_dfs)}")
    print(f"Expanded nodes: {expanded_nodes_dfs}")
    print(f"Maximum queue size: {queue_size_dfs}")
    print(f"Path: {path_dfs}")
    print("="*50)

    return (expanded_nodes_dfs, queue_size_dfs, path_dfs)
```

In [ ]:
```
result = run_maze_search()
print(f"\nReturned result: {result}")
```

```
Maze:
[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

==================================================
SHOWING RESULTS FOR BFS:
==================================================
Path length: 1.4142135623730951
Expanded nodes: 4
Maximum queue size: 6
Path: [(1, 0), (0, 1)]

==================================================
SHOWING RESULTS FOR DFS:
==================================================
Path length: 1.4142135623730951
Expanded nodes: 91
Maximum queue size: 49
Path: [(1, 0), (0, 1)]
==================================================

Returned result: (91, 49, [(1, 0), (0, 1)])
```